Literature Citation Verification System Based on DeepScientist

Project A — Engineering Track: Evidence Chain Module

Miaoyi Liao Xiaoyu Xiong Yuxiang Liu

STA304 Final Project · LLMs: From Principles to Applications

June 2026

The Problem: Trust Deficit in Research Agents

Research Agents Have a Credibility Gap

LLM-based research agents (DeepScientist, OpenClaw) produce **professional-looking reports** with citations — but users cannot answer three critical questions:

1. **Which conclusions are backed by real sources?** Citations are untraceable
2. **Which are model inferences or hallucinations?** No automated detection exists
3. **Which have no evidence at all?** Wording often stronger than evidence justifies

Real-World Consequence

In our baseline testing with unmodified DeepScientist: a single report contained **3 fabricated citations** that passed visual inspection. The agent confidently cited "Smith et al. 2023" for claims — but that paper does not exist in Semantic Scholar's database of 200M+ papers.

Why This Matters: The Hallucination Problem

The Scale of the Problem

- LLMs generate **3–27% fabricated references** in academic writing tasks (Walters & Wilder, 2023; Bhatt, 2024)
- In scientific peer review, **up to 69% of GPT-4 generated reviews** contain hallucinated citations (Liang et al., 2024)
- These fabrications are **indistinguishable from real citations** without external verification
- For research agents that autonomously generate proposals and ideas, this risk is amplified

Our Response

- **Every claim** extracted from a source paper is verified against a **real API** (not an LLM)
- **3-source fallback**: Semantic Scholar (200M+ papers) → arXiv (2.4M open-access) → Crossref (150M+ records)
- Results are **scored**, not estimated: verdict + confidence + evidence snippet
- Final report includes an **auditable evidence chain** that cross-references every citation

System Architecture: Before & After

Before: Original DeepScientist

`QQ User` $\rightarrow$ `Agent (LLM)` $\rightarrow$ `Report`

No verification — No evidence IDs
No audit — C+ grade

After: Our System

`QQ` $\rightarrow$ `crossdisc_idea (C)` $\rightarrow$ `parse_pdf (A)`

$\downarrow$

`verify_claim` $\times$N `(A)` $\rightarrow$ `score_batch (B)`

$\downarrow$

`render_report (B)` $\rightarrow$ `record+memory (C)` $\rightarrow$ `QQ`

+ 5 new modules — **+ 3 MCP namespaces**
+ 139 tests — **$\geq$ B- grade**

A (Tool): claim_extractor, semantic_verifier, HARD STOP RULE **B (Scoring)**: traffic_light, render, MCP 5-tool reg **C (Integration)**: crossdisc_idea, QQ evidence, memory schema

FactCheck Pipeline: 5 Phases, 6 Mandatory Tools

Phase 0 — PDF Parsing (A) PyPDF2 → regex detection of "REFERENCES" → split by `[N]` / `[Author,Year]` → Unicode quote title matching → truncation at 200K chars / 40 claims. Handles IEEE, APA, and named-author formats.

Phase 1 — Verify (A) ***323 calls across 6 quests*** (1) Semantic Scholar → (2) arXiv Title API → (3) Crossref DOI search. Returns verdict + confidence + evidence snippet.

Phase 2 — Score (B) ***7-branch RYG logic***

supported conf ≥ 0.7	• Green (correct)
contradicted conf ≥ 0.7	• Red (wrong)
all other combinations	• Yellow (uncertain)

Phase 3 — Render (B) RYG summary table + per-claim detail cards + Evidence Chain table (E00X ↔ C00X).

Phase 4 — Persist (C) ***36/36 calls*** `artifact__record` → evidence_store. `memory__write` → episodes.

Data Flow Claim → VerificationResult → ScoredClaimResult → FactCheckResult

Enforcement — 2 Layers L1: 5-State Machine (SKILL.md) with exit checks. L2: HARD STOP RULE (builder.py).

Evidence Chain: Making Every Claim Auditable

3 Evidence Types (E00X system)

Source	ID Format	Recorded Metadata
QQ Text	E001	text + SHA256
QQ Image	E001-img	dims + format + SHA256
QQ PDF	E001-pdf	pages + SHA256 + sidecar

Report Structure (6 sections) §1

FactCheck (RYG), §2 Evidence Chain (E00X↔C00X), §3–5 Bridges + hypothesis, §6 Caveats + verifier limits.

Audit: `audit_report()`

- Scans report for evidence ID patterns (E, E-img, E-pdf)
- Cross-checks cited IDs vs. `evidence_store`
- Detects fake IDs, unused evidence, bare claims
- Outputs PASS or WARN verdict

Three-Tier Annotation

[E001]	Evidence-supported (●)
Inferred	Logically derived
Needs Verif.	Insufficient evidence

Live Demo: What Actually Happens

User sends via QQ:

```
/crossdisc_idea
```
[Attaches: pFedAFM.pdf, 14 pages]

Bot processes (2–5 min):

1. Downloads PDF → extracts text via PyPDF2
2. Calls `parse_pdf` → **40 structured claims**
3. Calls `verify_claim` ×**85 times** across S2/arXiv/Crossref
4. Calls `score_batch` → **G17 Y18 R5** (FAIL)
5. Calls `render_report` → colored Markdown
6. Calls `artifact__record` + `memory__write`

Bot returns via QQ:

FactCheck Score: FAIL
● 17 correct ● 18 uncertain ● 5 wrong (40 claims)

5 Cross-Discipline Bridges:
Medical diagnosis → Autonomous driving → Climate modeling → FinTech → NLP

Full evidence chain table with E001–E040 ↔ C001–C040 mapping

Test Results: 6 Quests Across 6 Disciplines

Quest	Discipline	Paper	Claims	Verif.	Score
027	ML/Federated	pFedAFM (14pp)	40	85	G17 Y18 R5 FAIL
028	Medical Edu.	Subinternship Dir.	8	27	G1 Y6 R1 FAIL
029	Chemistry	DFT Metal Clusters	1	7	G1 Y0 R0 PASS
030	Optimization	SGD Convergence	40	105	G10 Y28 R2 FAIL
031	Physics	LIGO PEMcheck	40	94	G2 Y37 R1 FAIL
032	ML Survey	FL Survey	1	5	G0 Y1 R0 WARN

Aggregate Metrics Across All 6 Quests

139 tests all pass | **996 events** logged | **323 verify_claim** calls | **36/36** mandatory tools completed

6 disciplines: CS · Medicine · Chemistry · Mathematics · Physics · Survey

Agent Compliance Journey: From 0 to 36/36

Iter.	Approach	Result	Lesson
v1	SKILL.md suggests	0/6 tools	LLMs ignore "please"
v2	MUST + code examples	10/40 verify, 0 score	Agent still cuts corners
v3	5-State Machine + exits	Partial fix	Exits help but bypassable
v4	**HARD STOP RULE**	**36/36**	**Framework > Prompt**

Key Insight: Prompt-Level vs. Framework-Level Enforcement

SKILL.md is advisory — the agent can still choose to hand-write scores. builder.py's HARD STOP RULE is mandatory — injected into system context and cannot be ignored. **Key lesson**: agent design must distinguish advisory from mandatory constraints.

Failure Cases: What Breaks and How We Handle It

1. **Verifier False Positives (LIGO Q031)** C036 (GW190707 event) flagged ● contradicted. Semantic Scholar matched a different paper with overlapping author names. **Handling**: Report explicitly states "almost certainly a false negative — verifier matched wrong paper."
2. **Abstract-Only Verification Limits (SGD Q030)** C018/C031 (mathematical convergence proofs) unverifiable from abstracts alone. **Handling**: Claims marked ● uncertain; caveats note "full-text access required for theorem verification."
3. **PDF Parser Cross-Discipline Failure (Medical Q028)** Medical papers indexed via PubMed, not arXiv — `parse_pdf` returned 0 structured claims despite successful text extraction. **Handling**: Agent auto-fell back to `bash_exec` manual extraction; evidence table marked `extraction_method: bash_fallback`.
4. **DeepSeek API Version Incompatibility (early, fixed)** Claude Code $\geq$2.1.154 sends `system` role messages unsupported by DeepSeek — 80% of runs crashed. **Fix**: locked Claude Code to v2.1.153, added `--strict-mcp-config`, filtered system Python from MCP PATH.

Boundary Cases: Proving Robustness at the Edges

Case	Quest	What We Proved
Minimal (1 claim)	Q029	System correctly outputs PASS; no crash on sparse input
Max uncertainty	Q031	37/40 ●; honestly explains abstract-only limitation
Non-standard refs	Q031	LIGO astro-ph format; system marks as not_found, not hallucinated
Internal proofs	Q027/030	Theorems have no external citation target; correctly flagged
Multi-citation	Q027	`[20]--[24]` dash-range parsed into 5 individual markers
Fake ID	test	`audit_report` correctly identifies E999-img as non-existent

Coverage: Normal case (Q027, Q030) + Degraded case (Q028) + Edge case (Q029) + Negative case (fake IDs) + Cross-domain case (Q031)

Key Technical Decisions

Decision	Choice	Rationale
Base system	DeepScientist	QQ connector + MCP framework + skill loader
PDF parser	PyPDF2	Handles 90% of academic PDFs, already installed
Verifier API	Semantic Scholar	Free, 200M+ papers, no API key needed
Fallback chain	S2 $\rightarrow$ arXiv $\rightarrow$ Crossref	3 complementary indexes, different metadata
Scoring logic	7-branch RYG	All verdict $\times$ confidence combinations covered
Enforcement	Prompt + Framework	SKILL.md suggestive, builder.py mandatory

What We Did NOT Build (and Why)

We deliberately scoped out full-text semantic entailment (requires paid API access) and LLM-based claim extraction (would introduce circular dependency — verifying LLM output with another LLM).

Before / After: What Changed

Dimension	Before	After
Message Recording	QQ logs only	E00X store (SHA256 + metadata)
Citation Annotation	None	[E001] + RYG 3-color labels
Reference Verification	LLM-only (hallucination risk)	S2 + arXiv + Crossref API
PDF Evidence	Not supported	Pages + SHA256 + text sidecar
Audit Capability	None	`audit_report` fake-ID detection
Experiment Logging	None	`memory_write` episodes schema
Test Coverage	0 tests	**139 tests / 100% pass**
Cross-Discipline	Single domain	**6 disciplines** validated
Tool Chain	C+ (prompt only)	$\geq$ **B-** (independent tools)

Limitations & Future Work

Current Limitations

- Abstract-only: most ● would be resolvable with full-text access
- Verifier false positives: $\sim$5–10% ● from title-matching errors
- PyPDF2 weak on scanned/image-based PDFs (requires OCR preprocessing)
- Network-dependent: offline mode not supported
- No video/audio evidence support

Future Improvements

- Full-text verification via arXiv open-access PDF parsing
- Local model fallback: Ollama + sentence-transformers for offline mode
- Structured NLP pipeline: replace regex with transformer-based claim extraction
- Multi-connector: extend to WeChat, Discord, Telegram
- Result caching: avoid duplicate API calls for same (claim, paper) pairs

Team Contributions

Member & Role	Scope	Key Deliverables
Miaoyi Liao Person A: Tool Layer	4 new + 3 mod files $\sim$1,500 loc	claim_extractor, semantic_verifier (3-API chain), HARD STOP RULE, evidence-chain store + runner instrumentation
Xiaoyu Xiong Person B: Scoring	3 new + 2 mod files $\sim$1,200 loc	traffic_light (7-branch, 24 tests), factcheck_render, MCP 5-tool registration, **6 Quest execution + all debugging**
Yuxiang Liu Person C: Integration	3 new + 8 mod files $\sim$800 loc	crossdisc_idea SKILL.md (5-Phase SM + state machine), QQ evidence recording, memory schema, daemon+runner fixes, 16 tests

Cross-Collaboration: The Phase 5 Enforcement Story

A provided HARD STOP RULE (builder.py) — **mandatory**. B rewrote Phase 5 as 5-State Machine (SKILL.md) — **structural**. C defined memory/artifact schema + evidence chain template — **auditable**. Three layers, one goal: agent cannot bypass the toolchain.

Summary: From Promise to Proof

We built a system that verifies what LLMs claim.

Scale	**Rigor**	**Impact**
6 disciplines	139 tests / 100% pass	C+ $\rightarrow \geq$ B-
6 quests completed	323 API verifications	36/36 mandatory calls
996 events logged	3-source fallback chain	4 failure cases analyzed

The system transforms research agents from **unverifiable storytellers** into **auditable evidence processors**. Every claim has an ID. Every ID has a source. Every source is checked.

Code: `https://github.com/leoplasture/STA304_Final_Project` **Report**: `docs/Final_Project_Report.md`